Seminar 4

Object-Oriented Design, IV1350 Seminar 4

Lucas Burgos, lburgos@kth.se

16 may 2026

1. Introduction

This report presents my solution for Seminar 4 in the course Object-Oriented Design, IV1350. The solution is based on the Repair Electric Bike scenario and extends the program from earlier seminars with exception handling and design patterns. The focus of this report is to showcase my implementation of handling customer search errors with exceptions, notify users and developers when errors occur and use the Observer pattern to inform technicians and receptionists when repair orders are updated. In addition to this the program also uses Singleton and Strategy as two further GoF design patterns. This report explains how I worked when implementing these parts, describes the resulting code, and discusses the design choices using the assessment criteria for Seminar 4.

2. Method

I started from the code I had from Seminar 3 and kept the same overall MVC structure, apart from some changes made with reference to my received feedback from seminar 3.

Because the program from Seminar 3 already followed a layered MVC structure, I decided to continue on the same principle. Customer registry errors should be put into the integration layer, controller-level exceptions should be inside of the controller layer, user messages in the view and repair order notifications and discount logic in the model.

The first part I worked on was exception handling, since this affects the customer search flow. In the earlier version, a customer search that did not find anything could be handled in a simpler way, for example by returning no customer. For Seminar 4, I changed this so that exceptions are used to describe the error in detail. I added NoSuchCustomerException for the case where a phone number does not exist in the customer registry. I also added DatabaseFailureException for the case where the customer registry cannot be reached. Since the program does not use a real database, I simulated this by always throwing DatabaseFailureException when a specific hard-coded phone number is searched for.

The controller catches these exceptions and translates them into exceptions that fit the controller layer better. Since the view does not need to know how the customer registry works internally, a better solution is to translate the two exceptions into two other exceptions, like customerNotFoundException and OperationFailedException. It only needs to know whether the customer could not be found or if the operation failed. After creating the

exceptions, I updated the view so that it catches the controller-level exceptions and handles them close to the user interaction. If no customer is found, the user gets an informative message that says that no customer exists with the searched phone number. If the simulated database failure occurs, the user instead gets a more general message saying that the search could not be completed.

To handle developer information, I added ErrorLogger. When a technical exception is caught in the view, the exception is written to a log file. This separates user notification from developer notification. The user gets a simple and understandable message, while the developer can inspect the log file to see what exception occurred and where the error came from. I also made sure that the controller only changes the selected customer after a successful search, so that the program state is not changed if an exception is thrown.

The next step was to implement the Observer pattern. I chose RepairOrder as the observed object because this is an object that changes during the repair process. A repair order is updated when a diagnostic report is added, when proposed repair tasks are added, when the order is prepared for customer approval, and when it is accepted or rejected. Since these changes happen inside RepairOrder, it made sense that RepairOrder should also be responsible for notifying interested observers. To implement this, I created the RepairOrderObserver interface. This interface defines one method that is called when a repair order has been updated. I then created two classes that implement the interface: RepairOrderView and RepairOrderLogger. RepairOrderView prints the updated repair order to System.out, so that technicians and receptionists can see when something has changed. RepairOrderLogger writes the same kind of update to a file, so that repair order changes can also be saved. To follow the idea of the observer pattern, the observers should not call the controller or actively search for information. They should only receive updates from the observed object. This is why RepairOrderView and RepairOrderLogger only implement the observer interface and react when their update method is called.

I also considered how the observer references should be passed to the observed object. The observers are created in startup, registered in the controller, and then passed to a new RepairOrder when the repair order is created. This keeps the creation and connection of objects in the startup/controller part of the program, while the responsibility for notifying observers stays in the model object. In this way, the controller does not need to know exactly when every update happens inside RepairOrder.

To avoid exposing too much of the model object, I implemented RepairOrderSnapshot as immutable value data for observers. Observers receive a RepairOrderSnapshot instead of the mutable RepairOrder. The snapshot does not expose references to domain objects such as Customer, Bike or RepairTask. Instead, it copies the relevant data into immutable value records: CustomerData, BikeData and RepairTaskData. CustomerData contains the customer name, phone number and email. BikeData contains the brand, model, serial number and warranty end date. RepairTaskData contains the task description and cost. The total cost is stored as BigDecimal. This makes observer communication safer because views and loggers can read the update data but cannot modify the observed repair order or any domain object used inside it.

For the two extra GoF design patterns, I chose Singleton and Strategy. I used Singleton for CustomerRegistry because it represents one shared customer registry in this small program.

Since there should only be one simulated customer database, I made the constructor private and provided one shared instance through getInstance. This makes it clear in startup that the program uses the same customer registry instance instead of creating several separate registries. For the discount calculation I used Strategy because discount rules can vary. I created the DiscountStrategy interface with two implementations: NoDiscountStrategy and WarrantyDiscountStrategy. RepairOrder uses a DiscountStrategy when calculating the total cost instead of having one fixed discount rule inside the class. This makes the design more flexible, since new discount rules, such as a loyalty discount or a seasonal discount, can be added later without changing the main repair order logic.

To support the warranty discount, I added warranty information to Bike in the form of a warranty end date. In this implementation, the warranty dates are hard-coded when the sample Bike objects are created in CustomerRegistry. One bike is given a warranty end date one year in the future, while another bike has a warranty end date one year in the past. This makes it possible to demonstrate both cases: a bike that receives a warranty discount and a bike that does not.

WarrantyDiscountStrategy checks whether the bike connected to the repair order is still under warranty by calling isUnderWarranty on Bike. If the warranty is still valid, the strategy returns a discount. If the warranty has expired, it returns zero. I chose this solution because the discount is based on explicit domain data in the model instead of being guessed from unrelated information, such as a serial number. In a real system, the warranty date would probably be stored in a database or entered when the bike is registered, but for this seminar program the hard-coded sample data is enough to show how the strategy works.

Finally, I added and updated tests for the new behaviour. The tests cover the exception cases, observer notifications, logging behaviour, repair order state changes and discount calculation. I also updated the sample execution so that it demonstrates both error handling and the normal repair order flow. The sample execution first shows the missing customer case and the simulated database failure, and then continues with a successful customer search, creation of a repair order, diagnostic report, approval and acceptance.

3. Result

The result of the implementation is that the program from Seminar 3 has been extended with exception handling, the Observer pattern and two additional GoF design patterns.

The customer search flow has been changed so that errors are handled with exceptions instead of simpler return values. CustomerRegistry now throws NoSuchCustomerException when no customer exists for the searched phone number. It also throws DatabaseFailureException when the hard-coded phone number for simulating a database failure is used. This makes the error situations explicit in the integration layer.

The controller has also been changed to handle these integration-level exceptions. When CustomerRegistry throws NoSuchCustomerException, the controller catches it and throws CustomerNotFoundException instead. When CustomerRegistry throws DatabaseFailureException, the controller catches it and throws OperationFailedException. This means that the view receives exceptions that match its abstraction level and does not need to depend on details from the integration layer.

The view now catches the controller-level exceptions and handles them in different ways depending on the error. If the customer does not exist, the view shows a message to the user explaining that no customer was found with the searched phone number. If the database failure is simulated, the view shows a more general message saying that the customer search could not be completed. The technical information about the exception is written to repair-error-log.txt by ErrorLogger. This separates user-friendly output from developer information.

The Observer pattern has been added around RepairOrder. RepairOrder is now the observed object, since it is the object that changes when diagnostic information is added, when repair tasks are proposed, and when the order is accepted or rejected. The class now keeps a list of RepairOrderObserver objects and notifies them when the repair order changes.

A new interface, RepairOrderObserver, has been added. Two classes implement this interface: RepairOrderView and RepairOrderLogger. RepairOrderView prints updated repair orders to System.out, while RepairOrderLogger writes updated repair orders to repair-order-log.txt. This means that technicians and receptionists can be informed automatically when a repair order changes, without asking the controller for the order.

RepairOrderSnapshot has also been updated to fully protect encapsulation. Instead of sending the mutable RepairOrder object or references to Customer, Bike and RepairTask, RepairOrder creates a snapshot containing copied value data. The snapshot uses the immutable records CustomerData, BikeData and RepairTaskData, and stores the total cost as BigDecimal. The observers receive only this copied snapshot data, which is enough for printing and logging but does not allow them to change the actual repair order or its domain objects.

CustomerRegistry has been changed to use the Singleton pattern. It now has a private constructor and one shared instance that is returned by getInstance. This fits the program because the customer registry represents one shared simulated database. Startup now gets the registry through CustomerRegistry.getInstance instead of creating a new registry object.

The Strategy pattern has been added for discount calculation. A new DiscountStrategy interface defines how a discount is calculated. NoDiscountStrategy returns no discount, while WarrantyDiscountStrategy gives a discount when the bike is still under warranty. RepairOrder now uses a DiscountStrategy when calculating the total cost. This makes the discount behavior replaceable without changing the main repair order logic.

Bike has also been changed to contain warranty information. This is used by WarrantyDiscountStrategy to decide whether a warranty discount should be applied. Because of this, the discount is based on real domain data in the model instead of being hard-coded in the calculation.

The startup class has been updated to connect the new parts of the program. Main creates the controller with a WarrantyDiscountStrategy, registers RepairOrderView and RepairOrderLogger as observers, and then starts the sample execution in View. This means that the sample run demonstrates both the observer updates and the warranty discount.

The result can be seen in the sample execution. First, the program searches for a phone number that does not exist and prints a user-friendly error message. Then it searches for the phone number that simulates a database failure, prints another user-friendly error message,

and logs the technical exception. After that, the program continues with a successful customer search, creates a repair order, adds a diagnostic report and repair tasks, prepares the order for approval, and accepts it. During these steps, RepairOrderView prints automatic updates whenever the repair order changes.

Overall, the program has become more robust and more flexible compared to the version from Seminar 3. Errors are handled explicitly with exceptions, repair order changes are communicated using Observer, the customer registry is shared using Singleton, and discount calculation can be varied using Strategy.

Sample run (Shorted and visually pleasing). Full sample run is available on GitHub:

****

ERROR: No customer was found with phone number 0000000000.

0a. Missing customer search result: none

ERROR: The customer search could not be completed. Please try again later.

0b. Database failure search result: none

1. Found customer: Sara Lind

2. Found bike: Crescent Elina, SN-12345

RepairOrderView update: orderId=1, state=NEWLY_CREATED, totalCost=0

5. Repair tasks:

- Replace battery connector, cost=900

- Update motor controller firmware, cost=650

RepairOrderView update: orderId=1, state=NEWLY_CREATED, totalCost=1395

RepairOrderView update: orderId=1, state=READY_FOR_APPROVAL, totalCost=1395

RepairOrderView update: orderId=1, state=ACCEPTED, totalCost=1395

Repair Order

Order ID: 1

State: ACCEPTED

Created Date: 2026-05-17

Estimated Completion Date: 2026-05-20

Problem Description: Battery drains quickly and motor cuts out.

Diagnostic Report: Loose battery connector found and outdated firmware detected.

Total Cost: 1395

****

4. Discussion

The implementation in Seminar 4 improves the program from Seminar 3 by making error handling more explicit and by introducing design patterns where they solve design problems. The main areas from the course literature that are relevant for this seminar are exception handling, polymorphism, design patterns, coupling, cohesion and encapsulation.

The exception handling follows several important guidelines from the course literature. First, the exceptions are used for situations that are actual error conditions. A missing customer and a database failure are not part of the normal successful flow, so it is reasonable to represent them with exceptions instead of returning null. The use of checked exceptions also makes the error handling better, since the caller is forced to either catch the exception or declare it further. Another positive aspect is that the exceptions are located at the correct layer. The integration layer throws NoSuchCustomerException and DatabaseFailureException, which describe problems related to the customer registry. The controller catches these and translates them to CustomerNotFoundException and OperationFailedException, which are more suitable for the view. This follows the principle that a layer should not expose unnecessary details about other layers. The implementation also separates user messages and developer messages. The user receives short and understandable error messages through ErrorMessageHandler, while technical information is written to a log file by ErrorLogger. This matches the course literature's idea of informing users and informing developers.

A possible weakness in the exception handling is that the simulated database failure is hard-coded to a specific phone number. In my own opinion it is acceptable for now since there is no real database, but for a production system this needs to be fixed. Another limitation is that the logging is simple and uses plain file writing. A larger application would normally use a logging framework with log levels, timestamps, configuration and good error handling.

For the Observer pattern I have chosen RepairOrder as the observed object because all relevant state changes occur there. Observer references are registered during startup and passed to each new RepairOrder when it is created. Later updates are therefore published from the model object that actually changed, instead of being manually coordinated by the controller. When a diagnostic report is added, when repair tasks are proposed, or when the order is accepted or rejected the repair order itself changes. This gives RepairOrder high cohesion, because the class is responsible for both managing its own state and notifying observers when that state changes. The observers, RepairOrderView and RepairOrderLogger, receive updates automatically when the observed object changes. This reduces coupling between the observers and the controller since they do not need to ask the controller for updated repair order information. Instead of sending the mutable RepairOrder object directly to the observers, RepairOrder creates a RepairOrderSnapshot and sends that with the update. The snapshot now contains only copied value data instead of references to domain objects. Customer, Bike and RepairTask are not exposed to the observers. Their relevant data is copied into CustomerData, BikeData and RepairTaskData, and the total cost is stored as BigDecimal. This protects encapsulation because observers can read and present the update, but they cannot directly change the actual repair order or any domain object used by it. This also solves the use case where technicians and receptionists need to know when a repair order has been updated without exposing the full model object. The way observer references are passed to RepairOrder is also worth discussing. The observers are created in startup,

registered in the controller, and passed to each new RepairOrder. This works and keeps the observer creation outside the model. At the same time, it means that the controller has some responsibility for managing observers. In my opinion this is acceptable for now, but in a larger system it might create problems.

The Singleton pattern is used for CustomerRegistry. This is easy to motivate in the context of the assignment because the customer registry represents one shared simulated customer database. It prevents different parts of the program from accidentally creating separate registries with different customer data. At the same time, Singleton has disadvantages. It introduces global access to one shared object, which can increase coupling if used too much. It can also make testing harder because shared state can remain between tests. In my implementation the risk is limited because CustomerRegistry is small and mostly read-only after construction, but in a larger application this could be a problem as well.

The Strategy pattern is used for discount calculation because discount rules can vary. RepairOrder does not need to know exactly which discount rule is used. It only depends on the DiscountStrategy interface. RepairOrder works with the general DiscountStrategy type instead of one specific discount class. Because of this, different discount strategies can be used without changing RepairOrder. The class only expects something that can calculate a discount, not a specific discount type. WarrantyDiscountStrategy is also based on explicit domain data in Bike, namely the warranty end date. This implementation is useful because the discount rule is based on meaningful model data instead of being hard-coded or inferred from unrelated information. A limitation is that the current strategy is still very simple. It only supports one warranty discount rule. A real system might need several combined discounts, customer loyalty rules or campaign discounts. In that case, Strategy could be combined with another pattern, such as Composite, to support multiple discounts.

My solution also improves cohesion by keeping responsibilities separated. CustomerRegistry handles customer lookup, Controller coordinates use cases, RepairOrder owns repair order state, ErrorLogger handles developer logs, and RepairOrderLogger handles repair order update logs. This makes the program easier to understand than if all behaviour had been placed in the controller or view.

Overall, the implementation satisfies the main goals of Seminar 4. It uses exceptions to handle error cases, informs both users and developers, implements Observer with RepairOrder as the observed object, and uses Singleton and Strategy as additional GoF patterns. The main weaknesses are connected to the simplified assignment environment: the database failure is simulated with a hard-coded value, logging is basic, and Singleton would be less suitable in a larger system. Despite these limitations, I think that my solution uses the relevant object-oriented design principles and is a good improvement of the program compared to Seminar 3.

References

Lindbäck, L. (2026). A First Course in Object-Oriented Development: A Hands-On Approach. January 14, 2026 version.

IV1350 Seminar 4 task list

IV1350 Seminar 4 assessment criteria

Seminar 2, my own design in astah

Seminar 3, my own solution from the previous seminar

Github url: https://github.com/lucas17skapar/EBRS-kod.git

Declaration on the use of AI: I used ChatGPT to speed up my work and the report writing. The AI was used to help structure the code, write comments, remind me of concepts from the course literature and to suggest improvements. I also used AI to debate with my design choices. I verified the AI's output by comparing it with the Seminar 4 assignment instructions, the assessment criteria and the course literature.